README

Retro-capture: CLI edit-safety (`/diff`, `/checkpoint`, `/undo`)

Retro-captured 2026-06-16 vs HEAD `9e6e0458` (`9e6e0458f93b7e50ee6d47d8324b34f02e77b31c`). Fresh CURRENT run on the running System — explicitly a retro-capture on 2026-06-16, NOT original/historical evidence (§11.4.6 / §11.4.123). Backfills the §11.4.83 docs/qa gap (G7) for the CLI edit-safety REPL commands (`/diff`, `/undo`, `/checkpoint` in `helix_code/cmd/cli/main.go`).

What was exercised (real end-user CLI REPL path)

`/undo` (real `git revert HEAD`) and `/checkpoint restore` are DESTRUCTIVE (they mutate the working tree). To keep this stream's scope (docs/qa only) and NEVER touch the real source tree, the destructive commands were exercised in a throwaway **isolated temp git repo**; the non-destructive `/diff` and `/checkpoint create`/`list` were run against the live repo (read-only / snapshot).

File	Feature	Real result
`cli_diff_checkpoint.txt`	`/diff`, `/checkpoint create retro-evidence-snapshot`, `/checkpoint list` (live repo, non-destructive)	`/diff` printed the real working-tree diff (dirty constitution submodule pointer); `/checkpoint create` produced a real git-backed checkpoint id + label; `/checkpoint list` listed it back.
`cli_undo_isolated.txt`	`/undo` (isolated temp repo)	Real `git revert HEAD`: a "Revert …" commit was created and `sample.txt` reverted from 2 lines back to 1 line. WORKS.
`cli_checkpoint_restore_isolated.txt`	`/checkpoint create` + `restore`	**REAL DEFECT FOUND** — see below.

File	Feature	Real result
	(isolated temp repo), tracked-vs-untracked characterization	

REAL DEFECT FOUND (anti-bluff finding — reported, NOT fixed here)

`/checkpoint restore` prints an **unconditional** ✅ `working tree restored to checkpoint <id>` success line, but the git-backend checkpoint only snapshots & restores git-TRACKED content:

- **Tracked file** (`tracked.txt`, committed before checkpoint): create → bad edit → restore ⇒ CORRECTLY restored. WORKS.
- **Untracked file** (`work.txt`, never `git add`-ed): create → bad edit → restore ⇒ NOT restored (stayed corrupted). `git ls-tree -r <checkpoint-ref>` confirms `work.txt` was never snapshotted — yet restore still printed the green success message.

This is a §11.4 success-message-vs-reality mismatch bounded to untracked working-tree files: a user who checkpoints a tree with untracked files, edits them, then restores, gets a green "restored" message while those files stay corrupted. Root cause characterized in `cli_checkpoint_restore_isolated.txt`. Filed for the conductor to track; source fix is out of this stream's scope.

Anti-bluff / quiescence notes

- Destructive commands never touched the real source tree (isolated temp repo, removed after capture).
- Live-repo `/checkpoint create` produced a `refs/helix/checkpoints/...` ref (the feature's own durable git-backend data — a Jun-15 ref from another session pre-existed, so this namespace is established behaviour) and a transient `.helix/` working dir, which this stream removed for quiescence (§11.4.84). HEAD stayed `9e6e0458`; no source file modified.